Real-world (score an answer that isn't exact) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The order matters here more than in any other exercise in this document: **write your pass/fail rules before you look at a single real output.** If you write the rules after seeing the answer, you'll write rules that just describe whatever the model happened to say.

Two ways to check rules against text that varies:

- **Property checks in plain Python** — free, fast, no extra API call, but only for rules code can express: "contains the word X", "is under N words".
- **LLM-as-a-judge** — a second model call, given the task, the answer and your rules, asked for PASS/FAIL (or a 1-5 score) plus a reason. Needed for rules about meaning: "mentions the late delivery", "doesn't invent facts".

The judge is not a neutral referee. It's software you built, with a prompt, and it can be wrong. So before you trust it, test it: give it a few answers where *you* already know the verdict — a clearly good one, and clearly bad ones (off-topic, or confidently wrong) — and check it gets every one right. That small set is called a **calibration set**.

The exact pieces:

- A `call_model(prompt)` function using the OpenAI client from Doc04, with `temperature=0` — a judge should be as steady as possible, not creative.
- `build_judge_prompt(task, answer, rules)` — the rules go in as a list, built with a plain `for` loop.
- `score_output(task, answer, rules)` — calls the judge, splits the reply into the first line (the verdict) and the rest (the reason).

- A `@dataclass` `JudgeResult` with `verdict` and `reason`, so tests read `result.verdict`, not `result["verdict"]`.
- A saved `JUDGE_PROMPT_VERSION = "v1"`, so every score is tied to one exact judge prompt.
- For a 1-5 score: `int(first_line)`, wrapped in `try/except ValueError`, raising your own `JudgeParseError` — Doc01's named-error habit.
- A `CALIBRATION_SET` list of `{"answer": ..., "expected_verdict": ...}` dicts, and one test that loops over it.

Hint 2 — The plan, and almost the whole thing

```
JudgeResult(verdict, reason) as a dataclass
JUDGE_PROMPT_VERSION = "v1"

score_output(task, answer, rules) -> JudgeResult:
    lines = reply split at the first newline
    verdict = first line, stripped, upper-case
    reason = the second part if there is one, else ""

CALIBRATION_SET = [
    a clearly good summary          -> expected "PASS",
    an off-topic answer             -> expected "FAIL",
    a confident but wrong summary   -> expected "FAIL",
]

test_judge_calibration():
    for each case: score it, assert verdict == expected
```

Here's most of the calibration test — add one more clearly bad case yourself:

```
# practice/llm_output_testing_practice.py — Real-world section
CALIBRATION_SET = [
    {
        "answer": "Order #4411 arrived 6 days late due to a warehouse "
                  "delay; the customer wants the shipping fee refunded.",
        "expected_verdict": "PASS",
    },
    {
        "answer": "I like pizza.",
        "expected_verdict": "FAIL",
    },
    # your turn: a confident summary that gets the facts wrong
]
```

```
def test_judge_calibration():
    for case in CALIBRATION_SET:
        result = score_output(SUMMARY_TASK, case["answer"], JUDGE_RULES)
        assert result.verdict == case["expected_verdict"], result.reason
```

Full solution: Show me the solution